

CS & IT ENGINEERING

Programming in C

Arrays and Pointers

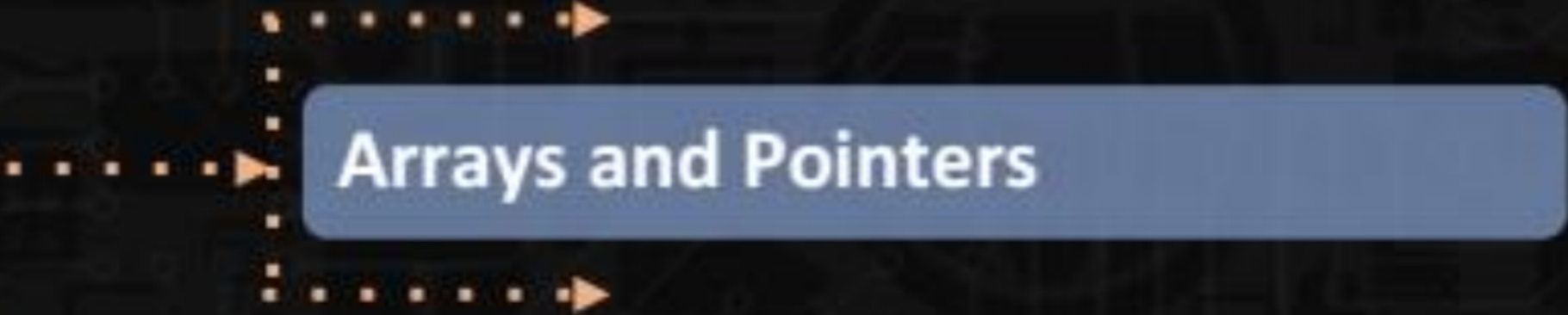
Lec- 08



By- Pankaj Sharma sir



TOPICS TO BE
COVERED



Arrays and Pointers

calloc

1) reliable

2) Expensive (time)

no. of blocks
size of each block

malloc(5 x sizeof(int))

calloc(5, sizeof(int))

✓ 1) Search for requested size of block.

① same

✓ 2) if found a block, it returns starting address of the block.

② it initialize all bits to 0.

③ returns the starting add. of that block

Garbage

0000- - - - 0

int *p;

~~realloc~~

p = malloc(5 * sizeof(int));

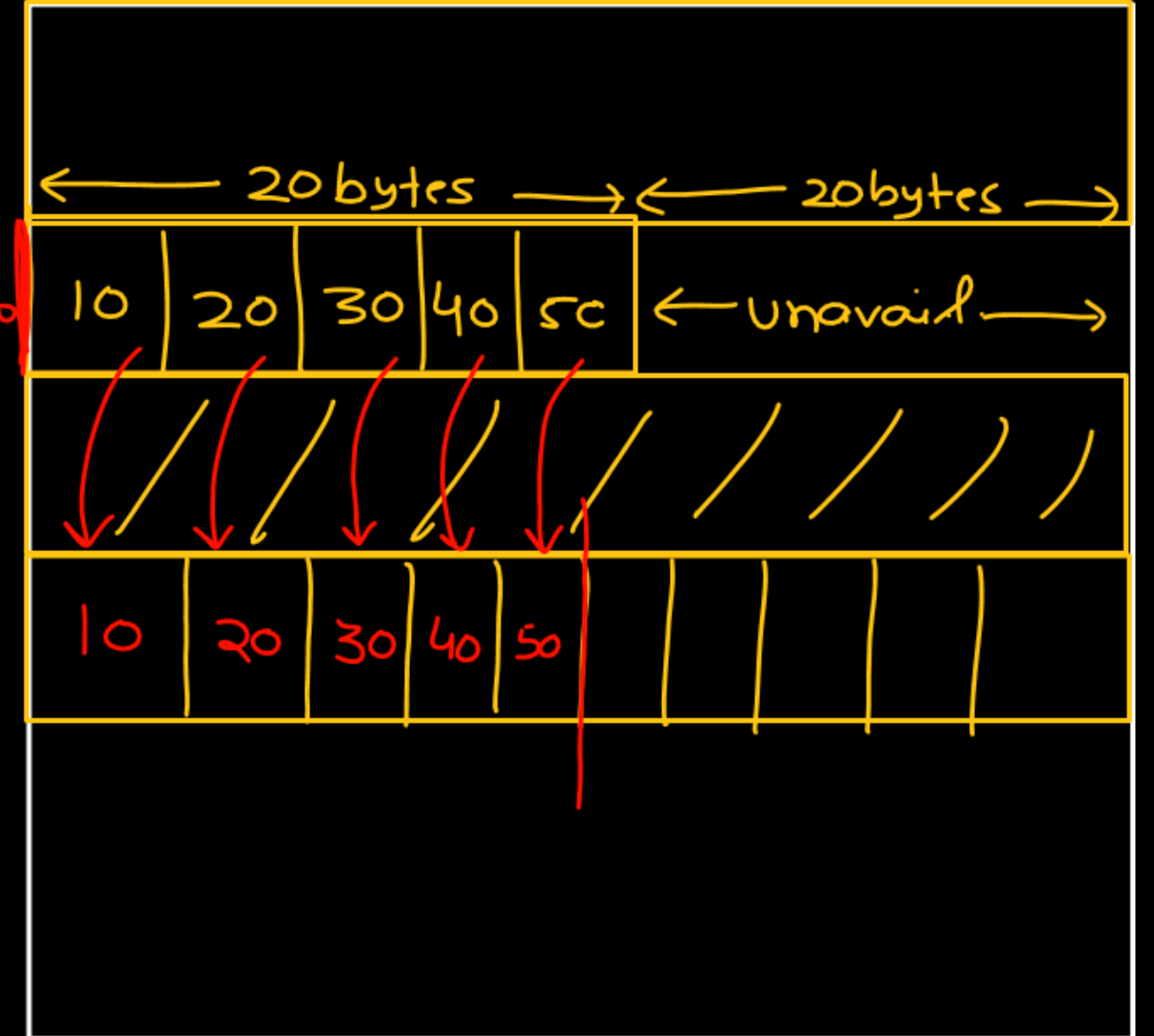
====>

realloc(p, new-size)

case 1

✓
p = realloc(p, 10 * sizeof(int));

~~q = realloc(p, 10 * —);~~

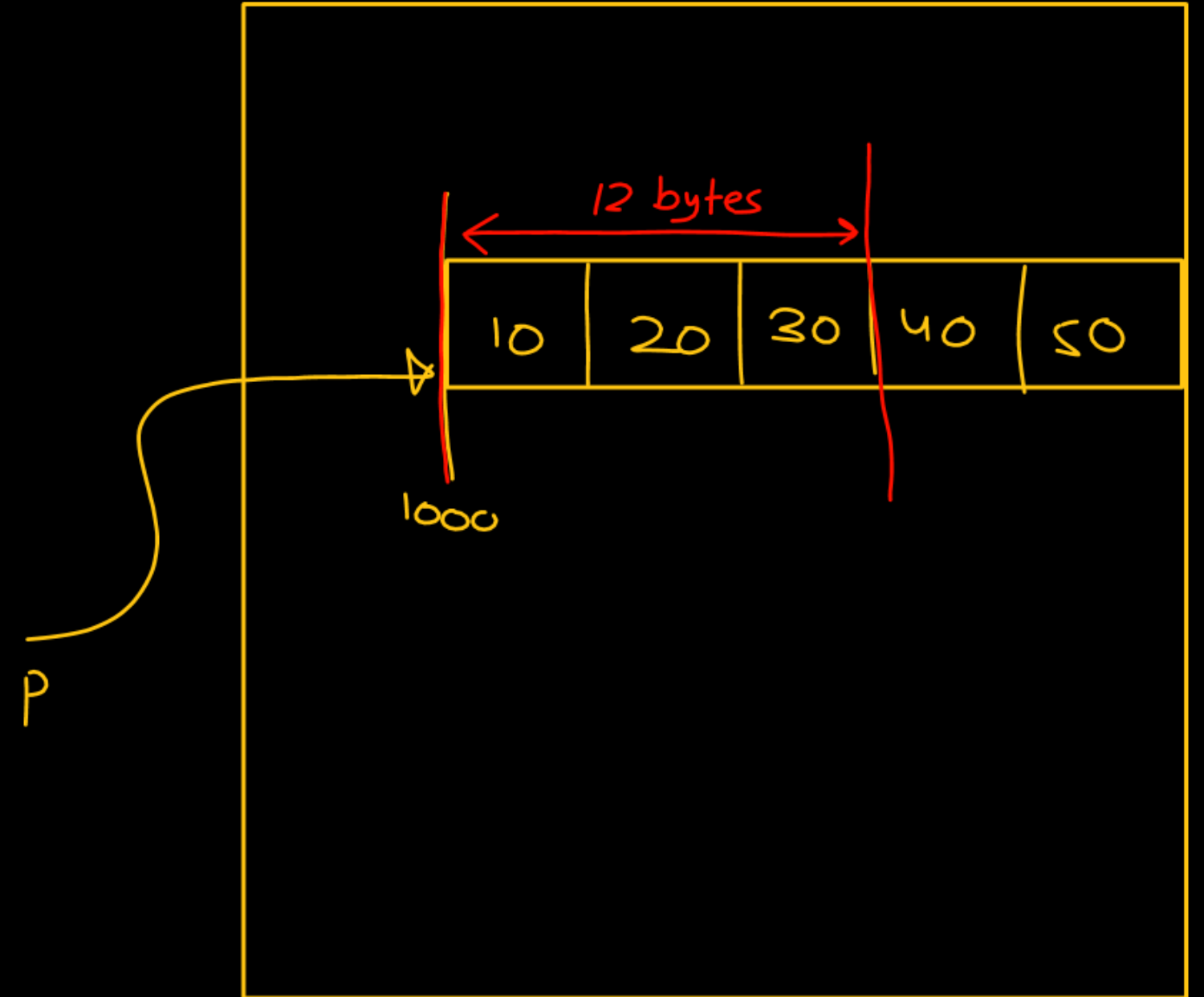


```
int *p;
```

```
p = malloc(5 * sizeof(int));
```

||
||
||
||

```
p = realloc(p, 3 * sizeof(int));
```



free

Exhausted

Memory leakage Problem

local variable

```
void main(){
```

```
    display(); ✓
```

```
    display();
```

```
    display();
```

```
    display();
```

```
    display();
```

```
}
```

```
void display(){
```

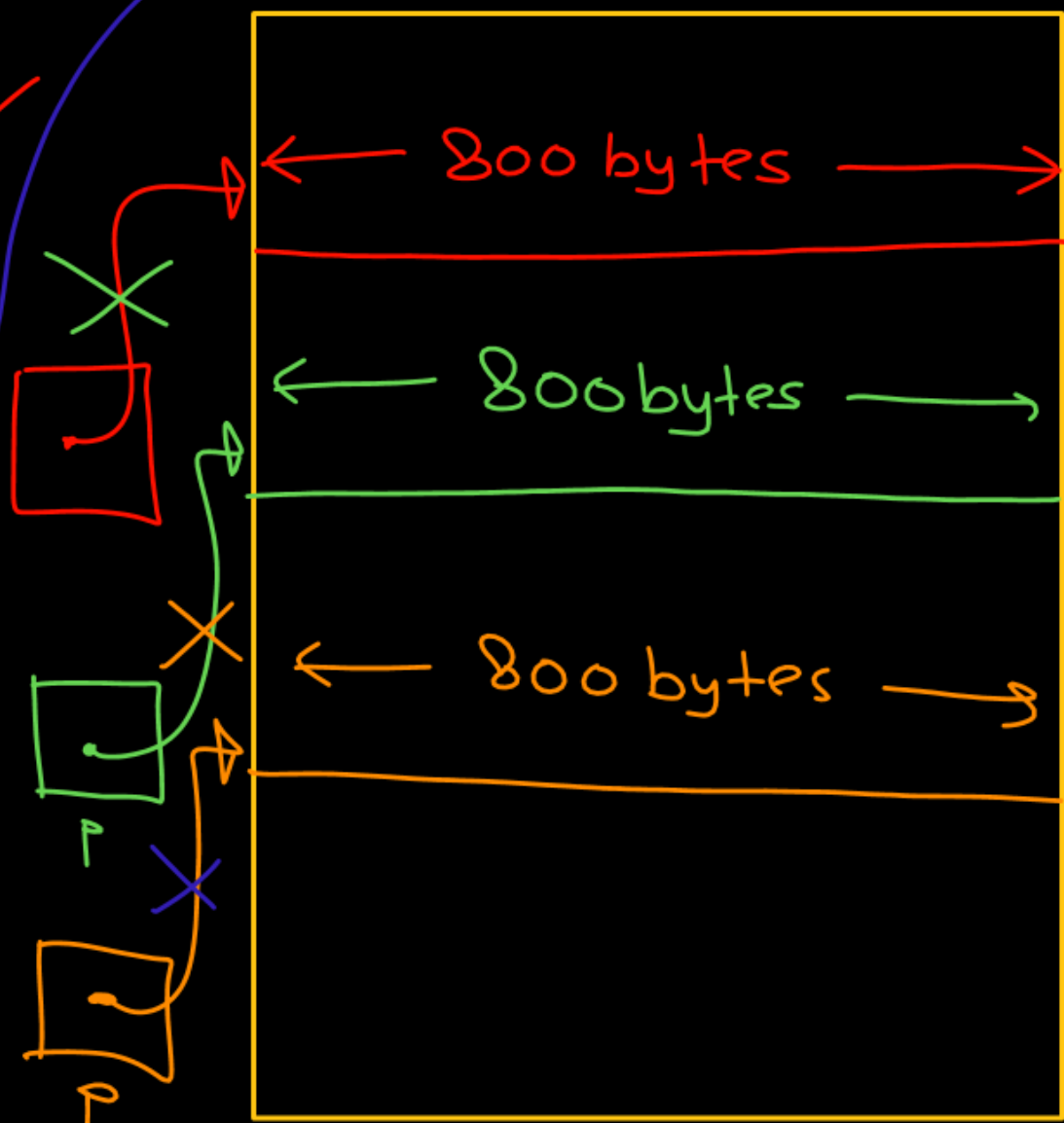
```
    int *p;
```

```
    p = malloc(200 * sizeof(int));
```

```
    |||
```

```
}
```

end



free

Exhausted

Memory leakage Problem

local variable

```
void main(){
```

```
display(); ✓
```

```
display();
```

```
display();
```

```
display();
```

```
display();
```

```
}
```

```
void display(){
```

```
int *p;
```

```
p = malloc(200 * sizeof(int));
```

```
||
```

```
||
```

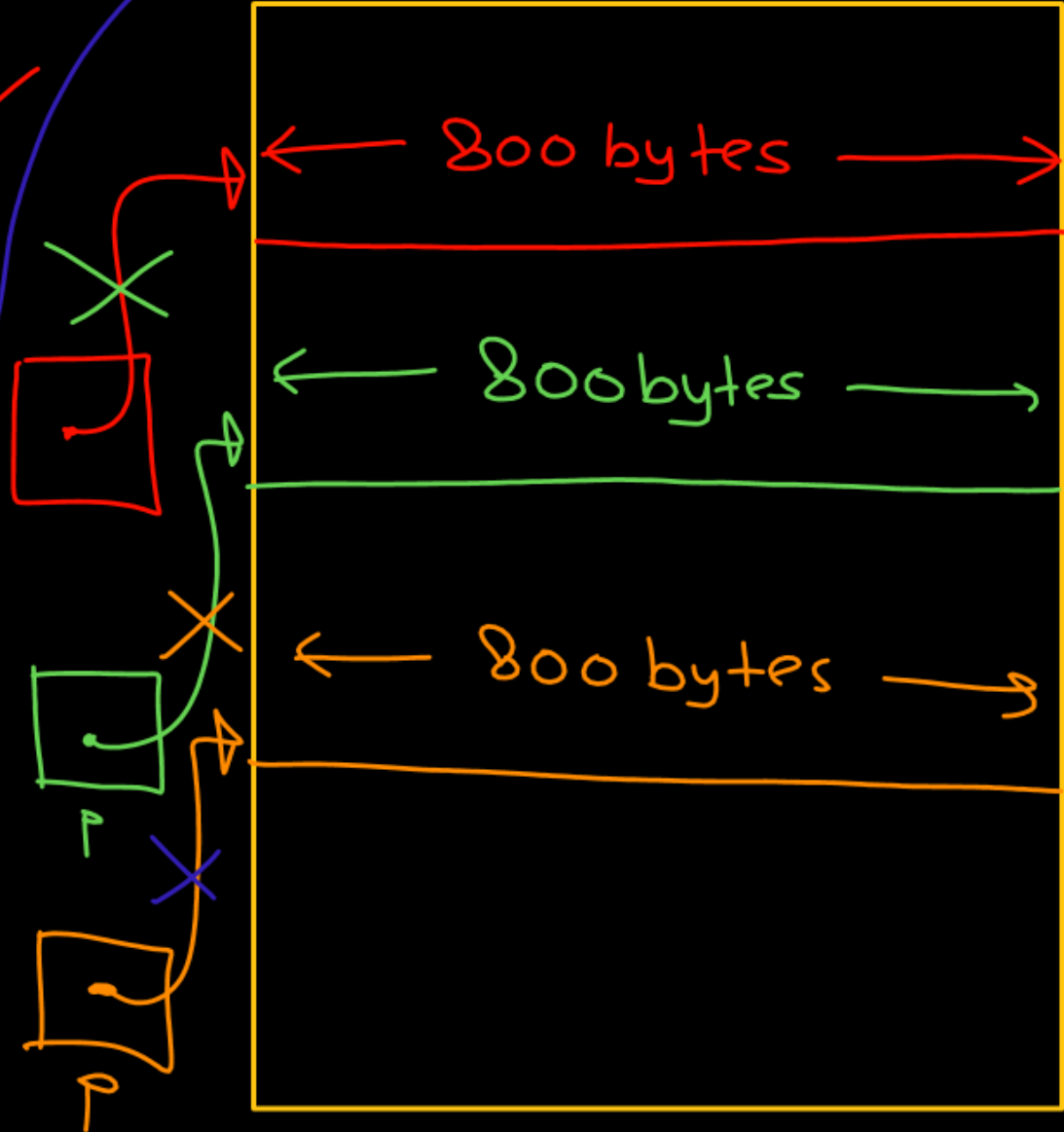
```
||
```

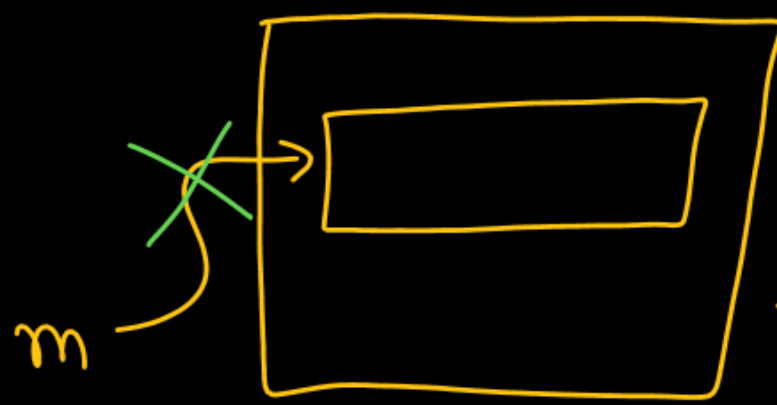
```
||
```

```
free(p);
```

```
}
```

end



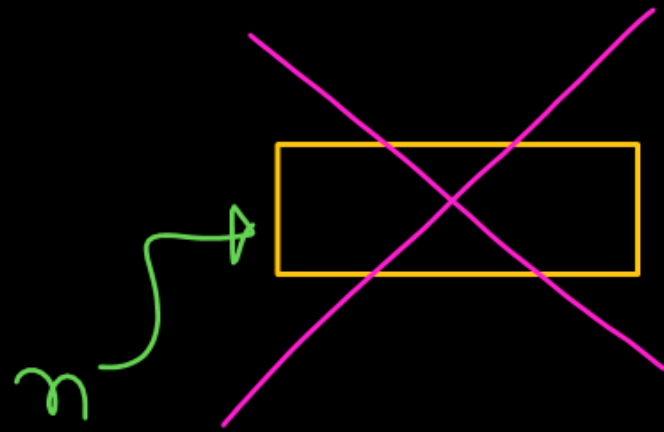


List-I

X: `m = malloc(5);`
`m = NULL;`

Y: `free(m);` ✓
`m → value = 5;`

Z: `char *p;`
`*p = 'a';`



List-II

1. Using dangling pointer

2. Using uninitialized pointer (wild pointer)

3. Lost memory

A) X-1, Y-3, Z-2

B) X-2, Y-1, Z-3

C) X-3, Y-2, Z-1

D) X-3, Y-1, Z-2

GATE
 - 2000


```
int * build_array();
```

```
int main(){
```

```
    int *a, k;
```

```
    a = build_array();
```

```
    for(k=0; k<5; k++)
```

```
        printf("%d", a[k]);
```

```
    return 0;
```

```
}
```

```
int * build_array(){
```

```
    int arr[5] = {1, 2, 3, 4, 5};
```

```
    return arr;
```

```
}
```

A) 1 2 3 4 5

☒ B) Segmentation fault

☐ C) Null pointer Exception

☐ D) None

```
#include <stdio.h>
#include <stdlib.h>
void main() {
```

```
    int i;
```

```
    int *ptr = (int*) malloc (5 * sizeof (int));
```

```
    for (i = 0; i < 5; i++)
```

```
        *(ptr + i) = i;
```

```
0 printf("/d", *ptr++);
```

```
1 printf("/d", (*ptr)++);
```

```
2 printf("/d", *ptr);
```

```
2 printf("/d", *++ptr);
```

```
3 printf("/d", ++*ptr);
```

```
}
```

A) C.E

~~B) 0 1 2 2 3~~

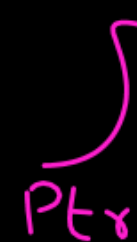
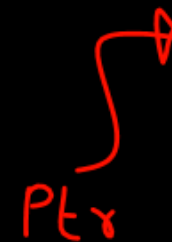
C) 0 1 2 3 4

D) 1 2 3 4 5

$++(*ptr)$

1) $*ptr = *ptr + 1$

2) Use $*ptr$



$\leftarrow ptr$

(i) $*ptr++ \rightarrow$ a) $*ptr$
b) $ptr = ptr + 1$

(ii) use $*ptr$ first
 $*ptr = *ptr + 1$

(iv) $*++ptr \rightarrow$ a) $ptr = ptr + 1 \rightarrow$
b) $*ptr$

```

int *assignval(int *x, int val)
{
    *x = val;
    return x;
}

```

```

void main() {
    int *x = malloc(sizeof(int));
    if (NULL == x) return;
    x = assign_val(x, 0);
    if (x) {
        x = (int*) malloc(sizeof(int));
        if (NULL == x) return;
        x = assign_val(x, 10);
    }
    printf("%d", *x);
    free(x);
}

```

A) C.E as the return of malloc is not typecasted properly.

~~B)~~ C.E because the comp. should be $x == NULL$

C) Compiled succ. but execution may result in dangling pointer.

D) Compiled succ. but execution may result in memory leak.

```

int *assignval(int *x, int val)
{
    *x = val;
    return x;
}

```

1000

$x_{\text{assign-val}}$

No way to access

```

void main() {

```

```

    int *x = malloc(sizeof(int));

```

```

    if (NULL == x) return;

```

```

    x = assignval(x, 0);

```

True Value

```

    if (x) {

```

```

        x = (int*) malloc(sizeof(int));

```

```

        if (NULL == x) return;

```

```

        x = assignval(x, 10);
    }

```

```

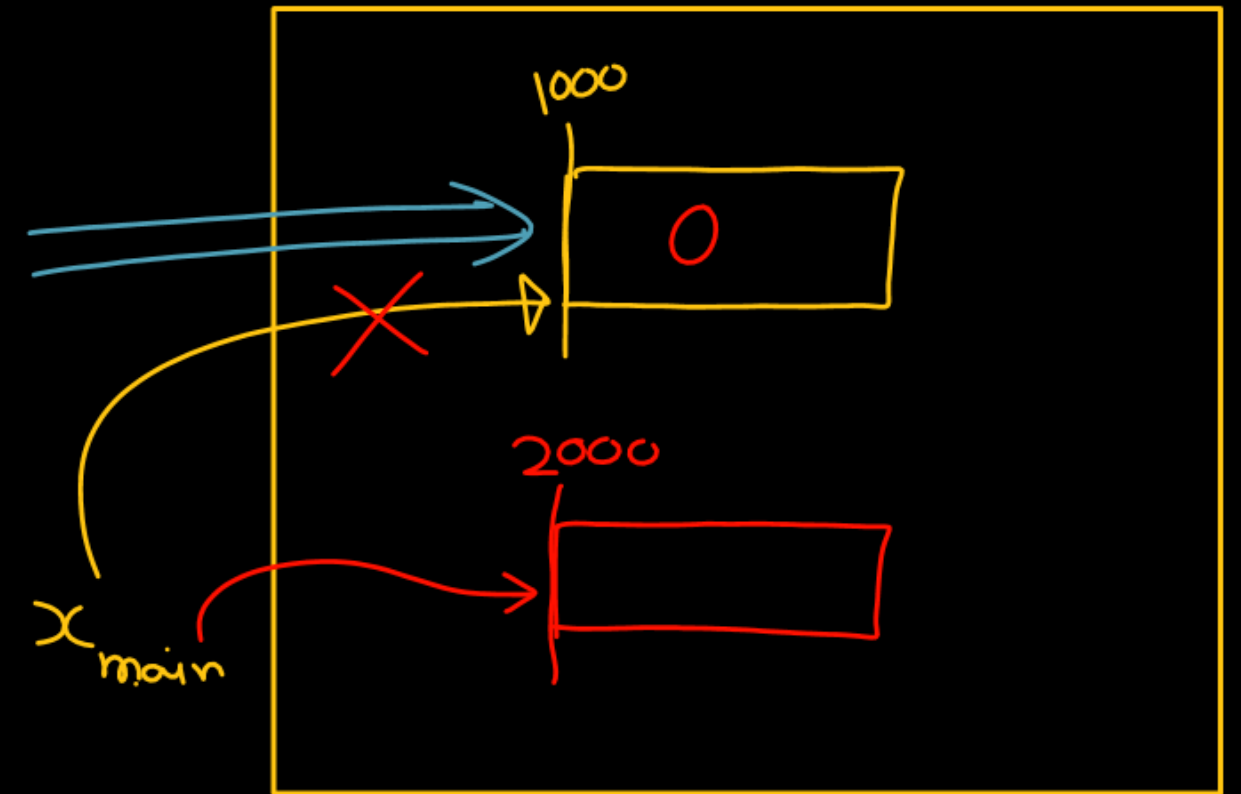
    printf("%d", *x);

```

```

    free(x);
}

```



[P1] `int *g() {
 int x = 10;
 return &x; } Problem`

[P2]

`int *g() {`

`int *px;`

`*px = 10;`

`Problem return px;
}`

Wild Pointer

A) Only P3

B) Only P1 and P3

C) Only P1 and P2

D) P1, P2 and P3

[P3]

`int *g() {`

`int *px;`

`px = (int*) malloc(sizeof(int));`

`*px = 10;`

`return px;
}`

A) $\text{Ptr} = \text{malloc}(a \times b)$

is same as

$\text{Ptr} = \text{calloc}(a, b)$

B) $\text{Ptr} = \text{calloc}(a, b)$ is same as

$\text{Ptr} = \text{malloc}(a \times b);$
 $\text{memset}(\text{Ptr}, 0, a \times b);$ ✓

$\text{malloc}, \text{calloc} \Rightarrow \text{void}^*$

THE
END

String

3:30 PM

